

ADVICE DOCUMENT

Written by ;
Ayomide Olagoke
Paul Demenko

Emmen, Netherlands
Version 1.0

Table of Contents

Version Control	3
Introduction.....	Error! Bookmark not defined.
UML Diagrams.....	Error! Bookmark not defined.
Classes	Error! Bookmark not defined.

Version Control

Version	Date	Content	Editor(s)
1.0	14/02/2024	First draft	Ayomide Olagoke
1.1	07/04/2024		Ayomide Olagoke

Introduction

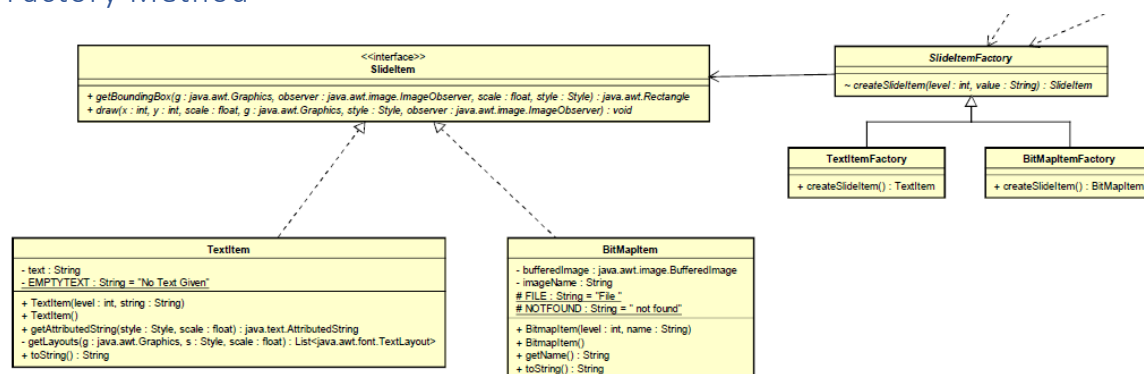
This document contains an in-depth explanation of design patterns used to improve the JabberPoint program, issues found in the initial version of JabberPoint programme, and the refactoring done to improve the readability and structure of the code.

We created UML diagrams to analyse the provided version of JabberPoint. We created an activity diagram to understand how JabberPoint runs, a class diagram to view the architecture of its code, a sequence diagram to show the process of running Jabber point, and a use-case diagram showing the functionalities expected by a user. We created new versions of these diagrams to give an overview of our proposed solution.

Design Patterns

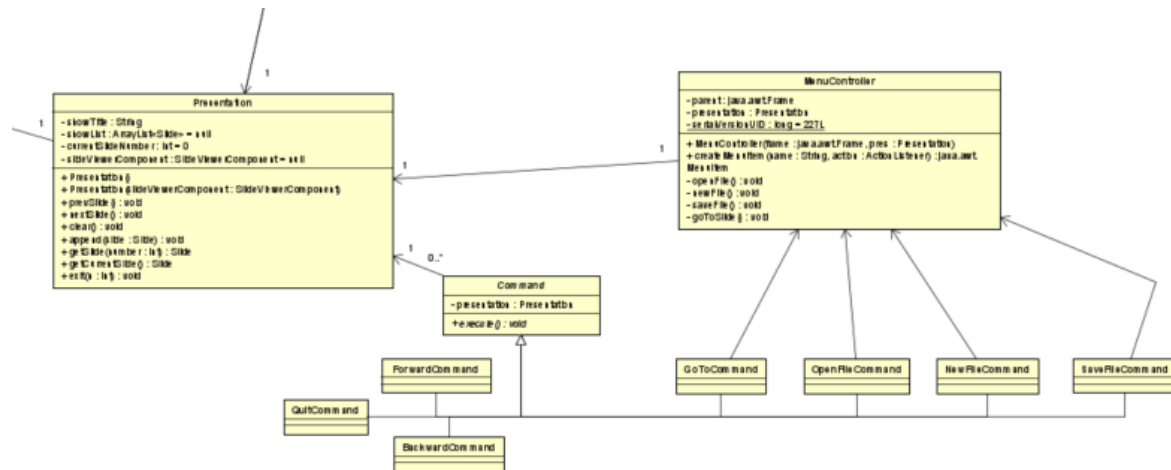
We applied two design patterns and noticed the presence of one design pattern in the program. We applied the factory method, creational design pattern, and command, a behavioural design pattern.

Factory Method



We applied this design pattern to the creation of slide items. This design pattern reduces the code required to construct a new slide item by abstracting the details of creation and encapsulating the creation logic within separate factory classes (`TextItemFactory` and `BitmapItemFactory`). By doing this, we improve loose coupling between the `Slide` class and the `SlideItem`-related classes. `SlideItem`-related classes can now be created by making the `Slide` dependant on using an instance of `SlideItemFactory` to create slide items. Using this design pattern allows the system to be open to adding a new type of slide item without modifying the code supporting the Open/Closed principle. We used factory method over Abstract factory (which can be applied in this case) to avoid unnecessary complexity.

Command



We applied this design pattern to initial design of the menu controller. We did it to turn every request into a standalone object, which gets carried out whenever the request is called. Instead of the `MenuController` containing the logic for all buttons/shortcuts related to requests, we created a `Command` abstract class with a method `execute()` that gets implemented by the `Command`-related classes. Using this design pattern promotes loose coupling and makes it easier to extend the system of requests. The support of asynchrony, flexibility, and efficient handling of user requests makes this design pattern a better choice over `State` or `Mediator` design patterns.

Refactoring

The initial version of the code contained issues such as;

- Bad comments,
- Poor code structure,
- Classes such as `MenuController`, `Slide` and others with methods with high cohesion,
- Non-self-explanatory variables, method parameters, and method names.

We refactored the program to contain understandable comments, applied design to promote the extension of certain functionalities, created methods to handle a single function in classes with high cohesion to reduce the present cohesion, added a util package that contains `Constants` class and a `ResourceAccessor` class that contains methods for loading files. We created unit tests to detect bugs and to ensure the program is working after refactoring. The commits are mostly self-explanatory, so every part of refactoring may be backtracked and understood.

DTAP

The initial git setup did not include any CI/CD or branching. We agreed to work on refactoring in a team, and we needed a setup to support that. The git repository consists of 3 branches – `main`, `test`, and `dev`.

main branch is used to release a stable version of the application. It has a pipeline that will automatically create a GitHub release whenever a pull request from test to main is merged.

test branch is running unit tests. It also runs code coverage and uploads the code coverage report to artifacts. The report is also hooked up to *CodeCov* to be inspected by the team. Sadly, the application cannot be run in a headless environment (and GitHub Actions do not have a connected display), so the report cannot be fully trusted. To evade this, the group is also submitting code coverage report generated by *IntelliJ*.

dev branch serves as the final endpoint for any feat branch. It runs basic compilation command to check that no broken code is merged. It also has a hook to a linter, *google-java-format*, that will check whether the files follow the Google's code conventions. If any mismatch is found, the pipeline will commit the changed files, so that only formatted files are present on dev branch.